

Academic Structure API – Frontend Guide

This guide documents the Academic Sessions, Classes, and Sections APIs.

Base URL and authentication

Base URL:

```
```text
http://sms.test/api
```
```

Every endpoint below requires a logged-in user token:

```
```http
Authorization: Bearer {token}
Accept: application/json
Content-Type: application/json
```
```

The backend finds the logged-in user's active institute automatically. Do ****not**** send `institute\_id` from the frontend.

Common response format

Successful requests use this shape:

```
```json
{
 "status": "success",
 "message": "...",
 "data": {}
}
```
```

Validation failures use Laravel's standard shape:

```
```json
{
 "message": "The given data was invalid.",
 "errors": {
 "field_name": ["Validation message"]
 }
}
```
```

If the authenticated user has no active institute, the API returns `422`.

1. Academic Sessions

Academic sessions represent a school year, for example `2026-2027`.

List sessions

```
```http
GET /institutes/academic-sessions
```
```

Create a session

```
```http
POST /institutes/academic-sessions
```

```

...

| Field | Type | Required | Notes |
| --- | --- | --- | --- |
| `name` | string | Yes | Example: `2026-2027`; unique within the institute. |
| `start_date` | date | Yes | Format: `YYYY-MM-DD`. |
| `end_date` | date | Yes | Must be after `start_date`. |
| `is_active` | boolean | No | Defaults to `true`. |

```

```

```json
{
  "name": "2026-2027",
  "start_date": "2026-08-01",
  "end_date": "2027-06-30",
  "is_active": true
}
```

```

### Get, update, or delete a session

```

```http
GET    /institutes/academic-sessions/{academic_session}
PUT    /institutes/academic-sessions/{academic_session}
PATCH /institutes/academic-sessions/{academic_session}
DELETE /institutes/academic-sessions/{academic_session}
```

```

Use the same fields as create for `PUT` or `PATCH`.

Session response fields:

```

```json
{
  "id": 1,
  "institute_id": 1,
  "name": "2026-2027",
  "start_date": "2026-08-01",
  "end_date": "2027-06-30",
  "is_active": true,
  "created_at": "2026-07-23T00:00:00.000000Z",
  "updated_at": "2026-07-23T00:00:00.000000Z"
}
```

```

---

## ## 2. Classes

Classes are permanent institute-level classes, for example `Grade 1`, `Grade 2`, or `Nursery`. A class is not directly attached to an academic session.

### List classes

```

```http
GET /institutes/classes
```

```

### Create a class

```

```http
POST /institutes/classes
```

```

```

Field	Type	Required	Notes

```

| Field           | Type    | Required | Notes                                                     |
|-----------------|---------|----------|-----------------------------------------------------------|
| `name`          | string  | Yes      | Example: `Grade 1`; unique within the institute.          |
| `display_order` | integer | No       | Use to control UI ordering. Minimum `0`; defaults to `0`. |
| `is_active`     | boolean | No       | Defaults to `true`.                                       |

```
```json
{
  "name": "Grade 1",
  "display_order": 1,
  "is_active": true
},
```
```

Do not send `code`. The backend generates it from the name, for example `Grade 1` becomes `GRADE-1`.

### Get, update, or delete a class

```
```http
GET    /institutes/classes/{academic_class}
PUT     /institutes/classes/{academic_class}
PATCH  /institutes/classes/{academic_class}
DELETE  /institutes/classes/{academic_class}
```
```

Class response fields:

```
```json
{
  "id": 1,
  "institute_id": 1,
  "name": "Grade 1",
  "code": "GRADE-1",
  "display_order": 1,
  "is_active": true,
  "created_at": "2026-07-23T00:00:00.000000Z",
  "updated_at": "2026-07-23T00:00:00.000000Z"
},
```
```

---

### ## 3. Sections

A section belongs to one class. For example, `Section A` and `Section B` can belong to `Grade 1`.

### List sections

```
```http
GET /institutes/sections
```
```

To list only a class's sections:

```
```http
GET /institutes/sections?class_id=1
```
```

### Create a section

```
```http
POST /institutes/sections
```
```

| Field      | Type    | Required | Notes                                                    |
|------------|---------|----------|----------------------------------------------------------|
| ---        | ---     | ---      | ---                                                      |
| `class_id` | integer | Yes      | ID of a class belonging to the current user's institute. |

```
`name`	string	Yes	Example: `Section A`; unique within the class.
`capacity`	integer	No	Between `1` and `1000`.
`display_order`	integer	No	Minimum `0`; defaults to `0`.
`is_active`	boolean	No	Defaults to `true`.
```

```
```json
{
  "class_id": 1,
  "name": "Section A",
  "capacity": 35,
  "display_order": 1,
  "is_active": true
}
```
```

Do not send `code`. The backend generates it from the name, for example `Section A` becomes `SECTION-A`.

### Get, update, or delete a section

```
```http
GET    /institutes/sections/{academic_section}
PUT    /institutes/sections/{academic_section}
PATCH /institutes/sections/{academic_section}
DELETE /institutes/sections/{academic_section}
```
```

For update, send `class\_id` as well as the fields you want to update. The section can only be moved to another class in the current user's institute.

Section response fields:

```
```json
{
  "id": 1,
  "class_id": 1,
  "name": "Section A",
  "code": "SECTION-A",
  "capacity": 35,
  "display_order": 1,
  "is_active": true,
  "created_at": "2026-07-23T00:00:00.000000Z",
  "updated_at": "2026-07-23T00:00:00.000000Z"
}
```
```

## Frontend flow

1. Create or select an academic session.
2. Fetch classes with `GET /institutes/classes`.
3. Create or select a class.
4. Fetch sections for that class with `GET /institutes/sections?class\_id={classId}`.
5. Create, edit, or delete sections using the selected class ID.